FARM ASSIST

A Project Report Submitted in Partial Fulfilment of the Requirements for

DATA STRUCTURES AND ALGORITHMS – 3 (25CS2103E)

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering
Academic Year 2026–2027

Submitted by
Varnika Vadapalli – 2520030020
Hamsini Cherukuri – 2520030016
Y. Minajja Reddy – 2520030352
Team Number: 20 | Section: 9

Under the Guidance of
Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering

KONERU LAKSHMAIAH EDUCATION FOUNDATION
Hyderabad, Telangana, India
2026–2027

DECLARATION

Declaration

We declare that the report entitled “Farm Assist” documents the work undertaken by Varnika Vadapalli (2520030020), Hamsini Cherukuri (2520030016), and Y. Minajja Reddy (2520030352) for Data Structures and Algorithms – 3 during academic year 2026–2027. The report describes the supplied Java implementation and explains its algorithms, assumptions, and limitations. The team should confirm the department’s required declaration wording and ensure that all submitted results reflect the final source build.

Varnika Vadapalli – 2520030020
Hamsini Cherukuri – 2520030016
Y. Minajja Reddy – 2520030352

CERTIFICATE

Certificate

This is to certify that the mini-project report entitled “Farm Assist” is submitted by Varnika Vadapalli (2520030020), Hamsini Cherukuri (2520030016), and Y. Minajja Reddy (2520030352), Section 9, Department of Computer Science and Engineering, in partial fulfilment of the requirements for Data Structures and Algorithms – 3 (25CS2103E), academic year 2026–2027.

Signature of the Guide: _____ Signature of the HOD: _____

ACKNOWLEDGEMENT

Acknowledgement

We express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for guidance and academic support. We thank the department and faculty for providing the learning environment in which we studied data structures, string algorithms, graph algorithms, approximation, randomized methods, and complexity analysis. We are grateful to our institution for access to academic resources. Finally, we thank our teammates and families for their cooperation and encouragement during the project and preparation of this report.

ABSTRACT

Abstract

Farm Assist is a Java console application that supports searching agricultural text and demonstrates algorithms in a practical domain. It loads .txt files from a local data directory and uses Knuth–Morris–Pratt (KMP) string matching to find case-insensitive whole-word or phrase matches. When an exact query fails, a dynamic-programming edit-distance routine compares the query with a fixed agricultural vocabulary and may suggest a nearby term. The irrigation module models a directed network with capacities and uses the Edmonds–Karp maximum-flow algorithm, based on BFS over a residual graph, to calculate maximum deliverable flow and identify a minimum cut. The complete source also includes Module 5 demonstrations of NP-completeness, a Vertex Cover approximation, image segmentation as minimum cut, project selection as maximum closure, and deadline scheduling as flow. Module 6 demonstrates randomized Quicksort, Miller–Rabin, reservoir sampling, hashing concepts, and sequential examples related to parallel algorithms. This report documents requirements, design, pseudocode, implementation, complexity, testing, limitations, and future work. Analytical results are distinguished from measurements that must be collected by running the source.

TABLE OF CONTENTS

Table Of Contents

Preliminary matter: Title Page; Declaration; Certificate; Acknowledgement; Abstract; Table of Contents; List of Figures; List of Tables; Abbreviations. Chapter 1: Introduction. Chapter 2: System Requirements. Chapter 3: System Design. Chapter 4: Implementation and Pseudocode. Chapter 5: Results and Testing. Chapter 6: Challenges and Limitations. Chapter 7: Conclusion and Future Work. Appendices: compilation checklist, algorithm invariants, viva questions, and references.

LIST OF FIGURES

List Of Figures

Figure 1.1 Farm Assist problem overview. Figure 3.1 High-level architecture. Figure 3.2 Crop-search workflow. Figure 3.3 KMP prefix-table concept. Figure 3.4 Whole-word boundary check. Figure 3.5 Edit-distance matrix. Figure 3.6 Irrigation graph. Figure 3.7 Edmonds–Karp workflow. Figure 3.8 Residual graph and minimum cut. Figure 3.9 Module 5 examples. Figure 3.10 Module 6 examples. Figure 4.1 Main-menu workflow. Figure 5.1 Test coverage overview. These are conceptual figures described in the text, not screenshots of a graphical interface.

LIST OF TABLES

List Of Tables

Table 2.1 Functional requirements. Table 2.2 Non-functional requirements. Table 2.3 Software requirements. Table 3.1 Data structures and purposes. Table 3.2 Irrigation nodes and edges. Table 4.1 Important methods. Table 4.2 Algorithm pseudocode variables. Table 5.1 Core test cases. Table 5.2 Boundary tests. Table 5.3 Complexity summary. Table 6.1 Limitations and remedies. Table 7.1 Future enhancements.

ABBREVIATIONS

Abbreviations

DSA: Data Structures and Algorithms; KMP: Knuth–Morris–Pratt; LPS: Longest Proper Prefix that is also a Suffix; BFS: Breadth-First Search; DP: Dynamic Programming; OSA: Optimal String Alignment; NP: Nondeterministic Polynomial time; PTAS: Polynomial-Time Approximation Scheme; FPTAS: Fully Polynomial-Time Approximation Scheme; FPT: Fixed-Parameter Tractable; API: Application Programming Interface; I/O: Input/Output. V denotes vertices, E denotes edges, and $O(\cdot)$ denotes asymptotic growth.

1. INTRODUCTION

Background and context

Farm Assist addresses the difficulty of finding agricultural information across separate documents. Farmers and students may need information about crops, soil, irrigation, pests, plant diseases, fertilizer, cultivation, and harvesting. A local searchable corpus can make existing information easier to retrieve. The system is an educational prototype: it searches text already collected by the user and does not independently verify the truth, currency, or suitability of the material.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

1. INTRODUCTION

Problem statement

The project addresses two algorithmic problems in an agricultural context. First, a user needs to find exact terms or phrases in a collection of text files, even when letter casing differs or a query contains a small spelling error. Second, an irrigation network has capacity limits that constrain how much water can travel from a source to an outlet. Farm Assist implements string matching and spelling correction for the first problem and maximum flow for the second.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

1. INTRODUCTION

Objectives

The objectives are to load local text files, search them efficiently, display unique matching sentences, suggest a nearby agricultural vocabulary term when appropriate, compute maximum flow in a directed irrigation graph, and report minimum-cut boundary edges. The complete source also demonstrates approximation, reductions to flow, randomized algorithms, and introductory parallel-algorithm concepts. The report explains the design, pseudocode, complexity, testing approach, limitations, and future development options.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

1. INTRODUCTION

Scope and assumptions

The program is a Java console application. Crop information is read from .txt files inside a directory named data. The irrigation graph is encoded as a fixed adjacency matrix, and its capacities do not come from sensors or a live water network. Module 5 and Module 6 are educational demonstrations. The supplied source does not implement a website, user accounts, a database, live weather integration, image-based disease diagnosis, or a graphical user interface.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

1. INTRODUCTION

Motivation and significance

The project connects abstract DSA concepts with a recognizable application domain. KMP demonstrates pattern preprocessing; edit distance demonstrates dynamic programming; Edmonds–Karp demonstrates BFS and residual networks; Vertex Cover illustrates approximation; and Module 6 introduces randomized methods. The agricultural setting makes it possible to discuss the value and limits of algorithmic support without claiming that an algorithm alone can replace expert agricultural judgement.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

1. INTRODUCTION

Intended users

The immediate users are students, instructors, and demonstrators studying algorithm design. A user chooses an option, enters a query or test value, reads the console output, and returns to the menu. Farmers could benefit from a future carefully reviewed version, but deployment would require trusted content, local-language support, usability testing, and clear source attribution. This report distinguishes coursework demonstration from validated agricultural advice.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

1. INTRODUCTION

Report organization

Chapter 1 introduces the problem and goals. Chapter 2 defines requirements, assumptions, and the development environment. Chapter 3 describes architecture and data structures. Chapter 4 explains the Java implementation and gives pseudocode. Chapter 5 presents analytical results and test scenarios. Chapter 6 discusses challenges and limitations. Chapter 7 summarizes conclusions and future work. References and appendices provide setup guidance and review questions.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Requirements overview

Requirements are derived from the supplied abstract and complete Java source. The application runs locally and uses standard Java libraries. It does not require a network connection or an external package manager. The requirements describe the behaviour of the provided implementation and should not be interpreted as claims about features planned for a future version.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Functional requirements: search

The system shall show a menu, discover .txt files in the data directory, sort the files by name, load their contents, accept a search query, reject empty queries, and allow the user to return using back or exit. It shall search exact terms with KMP before attempting spelling correction. Matching sentences shall be deduplicated and the output shall be limited to eight displayed sentences.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Functional requirements: irrigation

The system shall represent the water source, canals, fields, and outlet as vertices in a directed capacitated graph. It shall calculate maximum flow, report flow used on each positive-capacity edge, and identify original edges crossing the residual minimum cut. The reported units are litres per minute as indicated in the source code. The graph is a fixed illustrative model.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Functional requirements: Module 5

Module 5 explains P, NP, co-NP, NP-hardness, NP-completeness, reductions, approximation, PTAS, FPTAS, APX, FPT, and kernelisation. Its demonstrations include a Vertex Cover 2-approximation, image segmentation as a minimum-cut model, project selection as maximum closure, and deadline scheduling as flow. These examples illustrate models and do not form a general-purpose industrial solver.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

2. SYSTEM REQUIREMENTS

Functional requirements: Module 6

Module 6 demonstrates randomized Quicksort, Miller–Rabin primality testing, reservoir sampling, randomized hashing concepts, and parallel algorithm concepts. The sorting routine uses random pivots. Miller–Rabin accepts a candidate integer and number of rounds. Reservoir sampling accepts stream size and sample size. The reduction, prefix sum, and sorting examples are sequential illustrations rather than actual parallel implementations.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Non-functional requirements

The program should be understandable, portable, and efficient for small educational inputs. Search should ignore letter case and reject matches embedded in larger alphanumeric words. Missing folders and empty corpora should be reported. Invalid numeric input should be handled where the helper is used. The current system does not promise concurrent users, high availability, audit logging, or production-grade access control.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Hardware and software

A basic computer capable of running a compatible Java Development Kit is sufficient. Required items are the Java source, a JDK, a terminal, and a data directory containing text files. The source uses standard packages including java.io, java.util, and java.math.BigInteger. Memory usage depends on the total corpus size because the program loads document contents into memory.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Feasibility and constraints

Technical feasibility is high because the implementation uses established algorithms and standard Java libraries. Operational usefulness depends on having relevant, readable, trustworthy text files. Economic cost is low for a classroom prototype. Constraints include the fixed vocabulary, simple sentence splitting, fixed irrigation graph, and lack of external validation of agricultural content.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

2. SYSTEM REQUIREMENTS

Acceptance criteria

The program should compile, display its menu, search a valid corpus, reject empty queries, return exact matches, suggest close terms when the threshold allows, and compute a feasible maximum flow. Educational modules should accept valid inputs and preserve their stated invariants. Formal submission should include actual observations from running the final source; expected results in this report are not substitutes for execution records.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

3. SYSTEM DESIGN

Architecture overview

The system can be viewed as four conceptual layers: interaction, application control, algorithms, and data. The interaction layer prints prompts and reads input. The control layer dispatches menu choices. The algorithm layer performs string matching, edit-distance correction, maximum flow, approximation, and randomized demonstrations. The data layer contains text files, vocabulary terms, maps, arrays, and graph matrices.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Corpus representation

Text files are discovered from the data folder and stored in a LinkedHashMap associating each filename with its contents. Files are sorted before loading, so traversal order is predictable. Matching sentences are collected in a LinkedHashSet to suppress duplicates while preserving first-seen order. This design is clear for small corpora but uses memory proportional to the total text size.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Agricultural vocabulary

The fixed vocabulary contains terms such as rice, wheat, tomato, soil, water, irrigation, fertilizer, pest, disease, cultivation, harvesting, nitrogen, phosphorus, potassium, rainfall, compost, and sustainable. It supports spelling correction rather than full agricultural knowledge representation. A limited list is easy to inspect but cannot suggest words that are absent from the list.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

KMP design

KMP preprocesses a pattern into an LPS array. Each entry records the longest proper prefix that is also a suffix for a prefix of the pattern. When a mismatch occurs, the algorithm uses the LPS value to avoid rechecking text characters. The search runs in O(n+m) time for text length n and pattern length m, excluding storage for reported match positions.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Whole-word boundaries

After finding a character match, the program checks the character immediately before and after it. A boundary is accepted at the start or end of the text, or when the neighbouring character is not a letter or digit. This prevents a query such as pest from being accepted inside pesticide. The rule is useful for English text but is not a complete multilingual tokenizer.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

3. SYSTEM DESIGN

Edit-distance design

The spelling correction routine lowercases the query and compares it with each vocabulary word using a dynamic-programming matrix. Insertion, deletion, and substitution have unit cost, and an adjacent transposition is also considered. The routine permits distance one for queries of length four or less and distance two for longer queries. Equal-distance ties depend on vocabulary order.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Irrigation graph

The graph has seven vertices: Water Source, Canal A, Canal B, Field 1, Field 2, Field 3, and Farm Outlet. Source capacities are 20 to Canal A and 15 to Canal B. Canal A connects to Fields 1 and 2 with capacities 10 and 8. Canal B connects to Fields 2 and 3 with capacities 10 and 12. Field-to-outlet capacities are 10, 15, and 12.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Edmonds–Karp

The maximum-flow method initializes a residual matrix from the capacity matrix. BFS searches only edges with positive residual capacity and records parent links. The algorithm traces the path backward to find its bottleneck, reduces forward residual capacity, increases reverse residual capacity, and adds the bottleneck to the total. It repeats until the sink is unreachable.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Minimum cut

After maximum flow ends, a BFS from the source identifies vertices reachable through positive residual edges. An original edge crossing from a reachable vertex to an unreachable vertex belongs to the cut induced by that reachability set. Such edges explain capacity boundaries in the mathematical model; they do not diagnose a physical blockage or damaged canal.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Module 5 architecture

Module 5 combines theory text with small executable examples. The Vertex Cover routine selects an uncovered edge and adds its endpoints, then marks incident edges as covered. Image segmentation encodes label costs and neighbour penalties as capacities. Project selection represents prerequisites with a high-capacity dependency edge. Deadline scheduling connects jobs to allowable time slots.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

3. SYSTEM DESIGN

Module 6 architecture

Randomized Quicksort chooses a pivot randomly and partitions an array. Miller–Rabin uses random bases to test compositeness. Reservoir sampling maintains a fixed-size sample from a stream. The hashing example prints bucket assignments for a toy function. Reduction and prefix-sum examples illustrate concepts but execute sequentially.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

3. SYSTEM DESIGN

Design trade-offs

An adjacency matrix is easy to inspect for a seven-node graph but consumes $O(V^2)$ space and scans every possible neighbour. Loading the corpus once makes repeated queries simple but increases memory use. A fixed vocabulary is transparent but incomplete. These choices favour teaching clarity over production-scale flexibility.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Compilation and execution

The source file is FarmAssistStringMatching.java. Compile from the project directory with javac FarmAssistStringMatching.java and run with java FarmAssistStringMatching. The same working directory should contain a folder named data with .txt files. Running from another directory may cause the relative data path to fail.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Main menu

The complete source menu offers crop search, irrigation flow, Module 5, Module 6, and exit. A loop reads the selected string and a switch statement dispatches the appropriate method. Invalid options print a prompt. The main method owns the Scanner and closes it when the user exits.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

File discovery and reading

getDataFiles checks for data, filters case-insensitive .txt extensions, and sorts by filename. readFile reads lines through BufferedReader and appends newline characters. loadCorpus stores file names and text in a LinkedHashMap. Explicit character encoding and more detailed file-specific errors would improve portability.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

4. IMPLEMENTATION

KMP pseudocode: LPS

BUILD-LPS(pattern): allocate an array of pattern length; initialize length to zero and i to one. While i is smaller than the pattern length, compare the current character with the character at length. If equal, increase length and assign it to lps[i]. If unequal and length is nonzero, fall back to lps[length−1]. Otherwise assign zero and advance i. Return the array.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

KMP pseudocode: search

KMP-SEARCH(text, pattern): return an empty list for null text, null pattern, or empty pattern. Build LPS, then maintain text index i and pattern index j. Advance both on equality. When the whole pattern matches, check both boundaries, record the start if valid, and fall back using LPS. On mismatch, fall back if j is nonzero; otherwise advance i.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

KMP complexity

For text length n and pattern length m, preprocessing takes O(m) time and O(m) space. The scan takes O(n), giving O(n+m) overall, excluding the storage of match positions. Searching all files costs approximately the combined corpus length plus preprocessing for each query. The program performs another scan to collect matching sentences.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Edit-distance pseudocode

EDIT-DISTANCE(a,b): lowercase both words and create dp[m+1][n+1]. Initialize dp[i][0]=i and dp[0][j]=j. For each i and j, calculate the minimum of insertion, deletion, and substitution/match. If the final two characters are transposed, also consider dp[i−2][j−2]+1. Return dp[m][n].

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Spelling correction

findClosestWord trims and lowercases the query, calculates distance to each agricultural term, and retains the closest word. It applies the length-dependent threshold and returns null when no term qualifies. This simple nearest-candidate method does not consider frequency or meaning. A future interface could show several candidates and ask the user to choose.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

4. IMPLEMENTATION

Sentence extraction

The program splits text on whitespace following sentence-ending punctuation. It checks each sentence with KMP and gathers matching sentences in a LinkedHashSet. Up to eight unique sentences are displayed. The approach is simple but may mishandle abbreviations, decimal numbers, headings, and punctuation conventions in other languages.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Maximum-flow pseudocode

EDMONDS-KARP(capacity,s,t): copy capacity to residual and set total to zero. While BFS finds a path, calculate the minimum residual capacity along that path. Subtract that bottleneck from each forward residual edge and add it to each reverse edge. Increase total by the bottleneck. Return total and the final residual graph.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

BFS residual search

BFS marks the source visited, adds it to a queue, and repeatedly removes vertices. It scans every potential destination and enqueues unvisited vertices connected by positive residual capacity, recording each parent. It returns whether the sink was visited. With an adjacency matrix, each BFS costs $O(V^2)$.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Vertex Cover approximation

The demonstration visits edges in order. When an uncovered edge is found, both endpoints are added to the cover and every edge incident to either endpoint is marked covered. For standard loop-free undirected graphs, the maximal-matching argument gives a 2-approximation. The special self-loop case should not be used to extend that theoretical guarantee without further analysis.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Image segmentation reduction

The toy graph contains a source, four pixel nodes, and a sink. Terminal edge capacities encode costs associated with foreground and background labels. Neighbour edges penalize disagreement. A minimum cut chooses labels balancing pixel evidence and smoothness. The code explicitly describes this as a toy example, not a decoder or production vision system.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

4. IMPLEMENTATION

Project selection reduction

The example assigns positive values to projects A and B and a cost to dependency C. Positive values connect from the source, costs connect to the sink, and a high-capacity edge represents the requirement that selecting A requires C. Residual reachability reveals the selected set, and the method calculates its net value.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Deadline scheduling reduction

The scheduling graph connects the source to jobs, jobs to allowable time slots, and slots to the sink. Unit capacities ensure that each job is assigned at most once and each slot receives at most one job. Maximum flow equals the largest feasible number of jobs scheduled before their deadlines.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Randomized Quicksort

The routine selects a random pivot index, swaps the pivot to the end, partitions the subarray, and recursively sorts the left and right partitions. Expected running time is O(n log n); worst-case time remains $O(n^2)$. Randomness changes the execution path, not the requirement that the final array be correctly sorted.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Miller–Rabin

The routine handles small values and even composites, decomposes n−1 into 2^s times odd d, and tests randomly selected bases. A witness can prove compositeness; a positive outcome is reported as probably prime. Under standard assumptions, the false-positive probability for an odd composite is bounded by one quarter per independent round.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Reservoir sampling

The first k stream items fill the reservoir. For each later item at index i, the routine selects a random index from zero through i. If it is smaller than k, the new item replaces that reservoir entry. The algorithm uses O(k) memory and O(n) time for n stream items.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

4. IMPLEMENTATION

Randomized hashing

The hashing demonstration explains universal hashing and perfect hashing, then prints assignments for a toy function. The source warns that this example does not prove the selected function family is universal. It is not a cryptographic hash and should not be used for security-sensitive applications.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

4. IMPLEMENTATION

Parallel concepts

The parallel demonstration computes a sum, inclusive prefix sum, and sorted array using sequential code. It explains work, span, Brent’s theorem intuition, and limits on speedup. Because the example does not create concurrent tasks, it demonstrates concepts rather than measuring parallel performance.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Testing methodology

Tests should cover menu navigation, corpus discovery, exact search, case-insensitive search, phrase search, boundary handling, empty queries, spelling correction, no-match behaviour, flow calculation, invalid numeric input, and each educational demonstration. Record actual input, expected output, observed output, and pass/fail after running the final source. Do not present anticipated output as an executed experiment.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Crop-search tests

A positive test places a sentence containing irrigation in a text file and searches for that term. A case-insensitivity test searches for IRRIGATION. A boundary test verifies that pest does not match pesticide as a whole word. A spelling test enters a near spelling and checks the returned suggestion. A no-match test uses an unrelated query and expects a clear message.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Analytical maximum-flow result

The source can send at most 20 units to Canal A and 15 to Canal B. Canal A can forward 18 units, and Canal B receives only 15 units even though its outgoing edges can carry more. Thus total flow is at most 33 litres per minute. A feasible allocation is 10 through Field 1, 15 through Field 2, and 8 through Field 3, for a total of 33. This is an analytical result derived from the encoded capacities, not a recorded runtime measurement.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

5. RESULTS

Flow allocation validation

One feasible allocation sends 18 from the source to Canal A and 15 to Canal B. Canal A sends 10 to Field 1 and 8 to Field 2. Canal B sends 7 to Field 2 and 8 to Field 3. Fields send 10, 15, and 8 respectively to the outlet. All capacities and intermediate flow balances are respected. Since 33 is also the upper bound, this proves that the maximum is 33.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Module 5 tests

Test Vertex Cover on a graph with one edge, a graph with no edges, and a small graph with several edges. Confirm that every edge is covered and that the standard approximation guarantee is discussed only for the intended graph class. Confirm that the image segmentation example labels four pixels, project selection respects its dependency, and scheduling never assigns more jobs than available slots.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Module 6 tests

Quicksort should produce a nondecreasing array and preserve duplicates. Miller–Rabin should reject values below two and even composites greater than two. Reservoir sampling should return exactly k items for valid input. The sequential prefix sum for values 1 through 8 is 1, 3, 6, 10, 15, 21, 28, 36; the sum is 36. Random samples need not be identical between runs.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Complexity summary

KMP takes $O(n+m)$ time and $O(m)$ prefix storage. Edit distance takes $O(mn)$ time and $O(mn)$ space. Edmonds–Karp has the standard $O(VE^2)$ time bound, while matrix-based BFS scans $O(V^2)$ entries per search. Randomized Quicksort has expected $O(n \log n)$ and worst-case $O(n^2)$ time. Reservoir sampling uses $O(k)$ space and $O(n)$ time.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

5. RESULTS

Reproducibility

For a formal experimental report, record the Java version, operating system, source revision, input files, exact queries, graph configuration, and observed outputs. Randomized routines may produce different internal choices; evaluate their invariants rather than expecting identical random sequences. Timing and memory tables should be added only after measurements have been collected on a stated environment.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

6. CHALLENGES AND LIMITATIONS

Corpus quality

Search quality depends on the information present in the corpus. Missing, outdated, duplicated, or poorly structured files can lead to incomplete results. Sorting files provides predictable order but does not validate content provenance. A future version should record source, publication date, review date, language, and topic metadata for each document.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

6. CHALLENGES AND LIMITATIONS

Spelling correction limits

A fixed distance threshold cannot account for all typing errors or semantic differences. Equal-distance candidates are resolved by vocabulary order, and only one candidate is returned. Improvements could display several suggestions, use frequency or context, support synonyms, and allow confirmation before searching a corrected term.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

6. CHALLENGES AND LIMITATIONS

Sentence segmentation limits

Splitting after punctuation and whitespace is lightweight but may mishandle abbreviations, decimal numbers, headings, and lists. Results also omit source metadata. A future version could use a tested sentence tokenizer, preserve paragraph identifiers, and display filenames and provenance with every matching passage.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

6. CHALLENGES AND LIMITATIONS

Graph modelling limits

The flow result is valid only for the graph encoded in the source. A real network may have changing capacities, losses, pumps, direction constraints, schedules, and demand priorities. Physical use would require domain experts to validate nodes, units, capacities, and assumptions. The current result is not a diagnosis of real canal damage.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

6. CHALLENGES AND LIMITATIONS

Module 5 limits

The theory method is a concise summary rather than a formal proof. Flow reductions are small hand-built examples. The Vertex Cover guarantee relies on the standard assumptions for loop-free undirected graphs. These demonstrations explain modelling patterns but do not solve arbitrary industrial instances.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

6. CHALLENGES AND LIMITATIONS

Module 6 limits

Miller–Rabin is probabilistic; reservoir sampling uses a demonstration stream; the hash function is not for security; and parallel-style examples execute sequentially. These boundaries should remain explicit in the report and in the program output. The examples are suitable for teaching concepts but not for claiming production capability.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

6. CHALLENGES AND LIMITATIONS

Maintainability and reliability

The single class contains several responsibilities. Refactoring into separate corpus, search, flow, approximation, and randomized-algorithm classes would improve testing. Explicit text encoding, clearer file errors, automated tests, and configuration files would strengthen reliability. If a web interface is added, safe rendering and additional security requirements would apply.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

7. CONCLUSION AND FUTURE WORK

Conclusion

Farm Assist combines agricultural text retrieval and graph-based irrigation analysis with algorithm demonstrations. KMP provides efficient exact matching, edit distance supports approximate vocabulary matching, and Edmonds–Karp computes maximum flow and a minimum cut. Module 5 demonstrates approximation and reductions; Module 6 introduces randomized and parallel-algorithm concepts. The project's main contribution is educational, and practical usefulness depends on the quality of the corpus and the accuracy of the model.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

7. CONCLUSION AND FUTURE WORK

Future work: search

A future version could use structured crop records, source metadata, an inverted index, language-aware tokenization, synonym lists, and ranked correction suggestions. Search results should identify their source and review date. Agronomic content should be reviewed and should distinguish general educational information from location-specific recommendations.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

7. CONCLUSION AND FUTURE WORK

Future work: irrigation

The graph could be loaded from a configuration file and visualized with capacities, flows, and cut edges. Automated checks could verify flow conservation and compare cut capacity with total flow. Larger sparse graphs could use adjacency lists. Sensor integration would require calibration, missing-data handling, and engineering review before any operational use.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

7. CONCLUSION AND FUTURE WORK

Future work: software architecture

The program could be divided into CorpusRepository, CropSearchService, SpellingCorrectionService, FlowNetwork, ApproximationExamples, RandomizedExamples, and ConsoleController. Unit tests and continuous integration could compile the source and test algorithm invariants after changes. A build tool and configuration file would improve maintainability.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

7. CONCLUSION AND FUTURE WORK

Skills learned

The project supports learning Java, file I/O, arrays, collections, dynamic programming, string matching, BFS, residual graphs, maximum flow, approximation, randomized algorithms, and asymptotic analysis. It also encourages testing, documenting assumptions, handling invalid input, and distinguishing theoretical claims from measured results. Team members can deepen understanding by manually tracing each algorithm and explaining its correctness.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

APPENDIX A

Compilation checklist

Place FarmAssistStringMatching.java in a project directory and create a data subfolder beside it. Add plain-text .txt files with known sample content. Run javac FarmAssistStringMatching.java, then java FarmAssistStringMatching from that directory. Test all menu choices and record actual output. If the data folder cannot be found, verify the working directory before changing the source.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

APPENDIX A

Algorithm invariants

KMP uses the LPS table to preserve valid prefix overlap after mismatches. Edit-distance dp[i][j] stores the minimum cost for converting prefixes. Flow updates preserve nonnegative residual capacities and allow cancellation through reverse edges. Quicksort partitioning places elements on the appropriate sides of the pivot. Reservoir sampling maintains equal inclusion probability for items already seen.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

APPENDIX B

Viva preparation

Be prepared to explain why KMP is linear, what LPS represents, how word boundaries differ from substring matching, why edit distance uses dynamic programming, what reverse residual edges mean, how a minimum cut is found, why the flow result is 33, what the Vertex Cover approximation guarantees, why Miller–Rabin says probably prime, and why the parallel examples are not concurrent implementations.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

Review item	Verification question
Input	Were normal and boundary cases considered?
Algorithm	Is the invariant and stopping condition clear?
Output	Does the result agree with the model?

REFERENCES

Recommended references

Recommended foundations include Cormen, Leiserson, Rivest, and Stein, Introduction to Algorithms; Kleinberg and Tardos, Algorithm Design; Sedgewick and Wayne, Algorithms; and the official Java Platform API documentation for java.io, java.util, and java.math. Confirm the exact edition, publisher, year, and access date of references actually consulted before submission. Standard algorithm texts cover KMP, edit distance, maximum flow, approximation, randomized algorithms, and parallel analysis.

Implementation and verification: the behaviour in this section should be checked against the supplied Java source and stated input assumptions. Reviewers should trace input, data structure, algorithmic invariant, stopping condition, and output.

Testing note: prepare a small hand-worked example, record expected intermediate states, and compare them with an actual execution. For search inspect indices and boundaries; for dynamic programming inspect base cases; for flow inspect capacities and conservation; for randomized methods verify invariants rather than identical random choices.

Design implications

The design is suitable for an educational prototype but may not cover every real-world input. Assumptions are stated explicitly. Future extensions should have acceptance criteria and regression tests.

REFERENCES

Recommended algorithm references

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
[2] J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2005.
[3] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.
[4] Oracle, Java Platform API Specification for java.io, java.util, and java.math. Consult the version corresponding to the JDK actually used.
Confirm the exact edition, publisher, year, and access date of sources consulted before final submission. Standard algorithm texts cover KMP, edit distance, maximum flow, approximation, randomized algorithms, and parallel analysis.

Submission quality checklist: verify names and identifiers, review the guide’s preferred certificate wording, inspect page numbering, confirm that the final source compiles, and replace illustrative test descriptions with observed results after execution. Do not add invented timings or screenshots.

APPENDIX C

Suggested viva questions

1. What problem does KMP solve, and why is LPS needed?
2. How does whole-word matching differ from substring matching?
3. What are the base cases of edit-distance dynamic programming?
4. Why does edit distance require O(mn) time and space?
5. What is a residual graph, and why are reverse capacities updated?
6. How does BFS define the Edmonds–Karp path choice?
7. What does a minimum cut mean for the irrigation model?
8. Why is the flow result not a physical diagnosis?
9. What is the standard Vertex Cover 2-approximation guarantee?
10. Why does Miller–Rabin report “probably prime”?
11. How does reservoir sampling use O(k) memory?
12. Why are the parallel examples sequential illustrations?

Submission quality checklist: verify names and identifiers, review the guide’s preferred certificate wording, inspect page numbering, confirm that the final source compiles, and replace illustrative test descriptions with observed results after execution. Do not add invented timings or screenshots.

APPENDIX C

Demonstration checklist

Compile the final Java source and ensure the data directory is in the working directory. Use a small corpus with known content. Demonstrate exact search, case-insensitive search, a misspelling, and a no-match query. Run the irrigation network and explain its capacity upper bound. In Module 5, show one Vertex Cover instance and one flow reduction. In Module 6, show sorting, primality testing, and reservoir sampling. Explain random variation and the limits of sequential parallel-style examples. Keep the demonstration consistent with the submitted source.

Submission quality checklist: verify names and identifiers, review the guide’s preferred certificate wording, inspect page numbering, confirm that the final source compiles, and replace illustrative test descriptions with observed results after execution. Do not add invented timings or screenshots.